Data-Driven Academic Performance Analysis System

Comprehensive Project Overview for Team

EXECUTIVE SUMMARY

We are building an intelligent system that helps students understand their academic performance at a detailed level. Instead of just getting a final grade (like 75%), students will upload their syllabus and exam papers, and our system will:

1. **Automatically identify which specific topics they're weak in**
2. **Show how their performance is trending over the semester**
3. **Provide personalized study recommendations**
4. **(Optionally) Predict future exam scores using machine learning**

Why this matters: Students have all the raw data (exam papers + marks) but no tool to analyze it. We're building that tool.

PROJECT ABSTRACT

This project introduces a **Data-Driven Academic Analytics and Performance Intelligence System** designed to provide students with actionable, topic-level insights into their learning progress.

Traditional academic systems provide only final grades (e.g., "You scored 75%"). **Our system** goes deeper:

- Reads student syllabi and question papers using AI (multimodal LLM)
- Automatically maps exam questions to specific topics
- Analyzes student performance at the **topic level**, not just aggregate marks
- Identifies strengths and knowledge gaps
- Tracks improvement trends over time
- Generates personalized study recommendations

The core innovation: Most academic analytics tools assume someone has already manually tagged which topic each question covers. We **automate this step** using AI, then let students verify the mappings. This removes the manual tagging bottleneck that existing systems require.

Optional ML component: We also explore machine learning models (Linear Regression, Random Forest) to predict future exam performance based on topic-level patterns—but this is experimental and clearly labeled as such.

Outcome: Students move from "I got 75%, now what?" to "I'm weak in calculus and probability, strong in algebra, and improving overall. Here's what to study next."

PROBLEM STATEMENT

The Gap

Students take multiple exams throughout a semester. They know their final scores, but they **don't have a structured way to understand:**

- ❌ Which topics they consistently struggle with
- ❌ Whether their weaknesses are improving or getting worse
- ❌ What to prioritize studying for the next exam
- ❌ How their performance compares across subjects

This information is **hidden in their question papers and marks**—but no accessible tool extracts it automatically.

Why It Matters

- Students waste time studying without a data-backed strategy
- Teachers have no insight into class-wide topic weaknesses (not scope of this project, but a downstream application)
- The raw data exists (question papers + marks), but remains unused

Current Workaround

Students manually:

- Flip through old exams
- Try to remember which topic each question belonged to
- Mentally tally up their accuracy per topic
- Make educated guesses about what to study next

In practice: Almost nobody does this rigorously. The insight is lost.

HOW THE SYSTEM WORKS

User Flow (High-Level)

```
Student uploads syllabus PDF
        ↓
System extracts topics & creates a structured topic tree
```

```
        ↓
Student uploads exam question papers (PDFs)
        ↓
System automatically maps each question to relevant topics
        (Marks questions as: "This is 70% Algebra, 30% Trigonometry")
        ↓
Student VERIFIES the mappings
        (Human checks: "Yes, correct" or edits them)
        ↓
Student enters marks scored on each question
        ↓
System computes & displays:
  - Topic-wise accuracy (how well did you do on algebra vs trigonometry?)
  - Consistency scores (are you reliable or do you fluctuate?)
  - Performance trends (is algebra improving or declining?)
  - Readiness score (overall preparation level)
        ↓
System recommends: "Focus on trigonometry next week"
        ↓
(Optional) ML model predicts: "You'll likely score ~78% on the next exam"
```

KEY FEATURES (What We're Building)

Core Features (Must Have)

Feature	What It Does	Why It Matters
Syllabus Parser	Reads PDF syllabus, extracts topic structure automatically	Replaces manual topic entry
Question-to-Topic Mapper	AI reads exam question, suggests which topics it covers	Automates the most tedious manual step
Human Verification Screen	Student reviews AI suggestions, approves or edits them	Ensures accuracy; compensates for AI mistakes
Mark Entry Interface	Simple form to enter marks per question	Minimal friction for data input
Topic Analytics Dashboard	Shows topic-wise accuracy, consistency, trends over time	The core insight students need
Readiness Scoring Engine	Rule-based transparent formula calculating overall readiness	Primary metric (transparent, not a black box)

Feature	What It Does	Why It Matters
Recommendation Engine	AI suggests personalized study focus areas	Actionable guidance based on data

Optional/Experimental Features

Feature	What It Does	Status
ML Prediction Module	Trains Linear Regression + Random Forest on topic patterns to predict future scores	Experimental (trained on synthetic data, clearly labeled)
Exploratory Data Analysis (EDA) Dashboard	Visualizations of performance distributions, correlations, historical patterns	Secondary; supports analysis but not primary output

---

PROJECT SCOPE

✅ IN SCOPE (We Will Build)

- Single-student system (one user at a time, not multi-user/classroom)
- PDF input only (syllabi and question papers)
- Manual mark entry per question
- One syllabus format (your university's standard format)
- Topic-level performance analysis (not question-level)
- Rule-based transparent scoring (primary metric)
- Experimental ML module (secondary, exploratory)
- Web-based interface (frontend + backend)

❌ OUT OF SCOPE (We Will NOT Build)

- Multi-student or classroom/teacher views
- Multi-institution syllabus format generalization
- Handwritten mark OCR (automatic extraction from marked papers)
- Mobile app (web-first only)
- Production-grade authentication (basic login only)
- Real-time collaboration or multi-user features

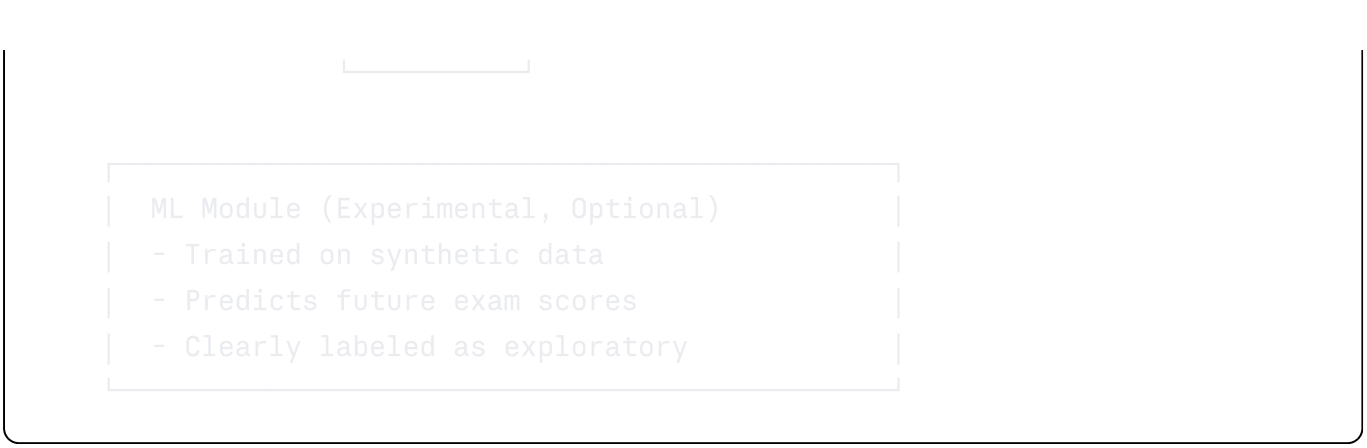
🚀 FUTURE WORK (After This Project)

- Expand to multiple syllabus formats

- Handwritten annotation OCR
- Teacher dashboards (class-wide topic weakness patterns)
- Validated ML model (with real longitudinal data from a pilot class)
- Spaced-repetition scheduling (tell students *when* to review)
- Mobile app

SYSTEM ARCHITECTURE (Simplified)





TECHNOLOGY STACK (Preliminary)

Frontend

- **Framework:** React.js (or Vue/Svelte)
- **Styling:** Tailwind CSS
- **Visualization:** Recharts, D3.js, Plotly
- **PDF handling:** PDF.js, react-pdf

Backend

- **Language:** Python (Flask/FastAPI) or Node.js (Express)
- **PDF/Document Processing:** PyPDF2, pdfplumber, or similar
- **Multimodal LLM:** Claude 3.5 Sonnet API or GPT-4V (for document parsing)
- **Database:** PostgreSQL (or SQLite for dev)

ML Components (Optional)

- **Libraries:** scikit-learn (Linear Regression, Random Forest)
- **Data Processing:** pandas, numpy
- **Validation:** cross-validation, metric reporting

Deployment

- **Backend:** AWS EC2 / Heroku / Railway (to be decided)
- **Frontend:** Vercel / Netlify
- **Database:** AWS RDS / Heroku Postgres

PROJECT PLAN & PHASES

Phase 1: Foundations (Weeks 1-2)

Goal: Set up structure and validate core pipeline

- [ ] Define database schema (syllabi, topics, questions, marks, mappings)
- [ ] Set up backend API structure (routes, models, authentication)
- [ ] Set up frontend boilerplate (React app, routing, basic layout)
- [ ] Test multimodal LLM on sample syllabus PDF
  - Verify it can extract topics correctly
  - Identify failure modes and edge cases

Deliverable: Working backend + frontend skeleton; LLM working on test PDFs

---

Phase 2: Core PDF Pipeline (Weeks 3-4)

Goal: Syllabus → Topics & Papers → Questions automatically

- [ ] **Syllabus Parser Module**
  - LLM reads PDF, extracts topic structure
  - Creates structured topic tree in database
  - API endpoint to upload & parse syllabus
- [ ] **Question Mapper Module**
  - LLM reads question paper PDF
  - Extracts individual questions
  - Suggests topic mappings for each question (weighted, multi-topic)
  - API endpoint to upload & parse papers
- [ ] **Verification Screen UI**
  - Frontend shows AI-suggested topic mappings
  - Student can approve, edit, or reject each mapping
  - Save verified mappings to database

Deliverable: End-to-end pipeline working: upload syllabus → extract topics; upload paper → extract & map questions; human verifies

---

Phase 3: Analytics Engine (Weeks 5-6)

Goal: Turn marks + mappings into insights

- [ ] **Scoring Engine** (Rule-based, transparent)
  - Topic accuracy = % correct on questions mapped to that topic
  - Consistency score = variance in performance across assessments
  - Recency weight = recent exams weighted higher than old ones
  - Readiness score = formula combining above (clearly documented)
- [ ] **Trend Analysis**
  - Track topic performance over multiple exams

- Show improvement/decline per topic
- Visualize on dashboard

☐ **Recommendation Engine**

- Identify weak topics (below threshold)
- Generate personalized study advice using LLM
- Rank recommendations by impact (easiest to improve first? weakest areas? your choice)

☐ **Dashboard UI**

- Leaderboard of topics (best to worst)
- Trend graphs
- Recommendation cards
- Readiness score display

Deliverable: Full analytics dashboard; student can view all insights

---

Phase 4: ML Module & EDA (Weeks 7-8)

Goal: Experimental prediction & exploratory analysis

☐ **Feature Engineering**

- Extract features: topic accuracy, consistency, recency weight, improvement rate, etc.
- Validate feature approach against OULAD dataset (public academic analytics dataset)

☐ **ML Models** (Experimental)

- Train Linear Regression on synthetic data (predict exam score)
- Train Random Forest on synthetic data
- Compare predictions to actual rule-based readiness score
- Clearly label as "Experimental/Exploratory"

☐ **EDA Dashboard**

- Correlation heatmaps (which topics correlate with exam success?)
- Distribution charts
- Hypothesis testing visualizations

Deliverable: ML prediction working; EDA charts in dashboard; disclaimer added to UI

Phase 5: Testing & Refinement (Weeks 9-10)

Goal: Make sure everything works on real data

☐ **Unit Tests**

- Test scoring engine formulas
- Test LLM parsing on various syllabus formats

- Test recommendation logic

☐ **Integration Tests**

- Full end-to-end flow: upload → parse → verify → enter marks → view dashboard

☐ **Real Data Testing**

- Test on your actual university's syllabus + past question papers
- Verify LLM mapping accuracy
- Refine prompts based on failures

☐ **UX Polish**

- Improve UI based on feedback
- Add error handling
- Optimize performance

Deliverable: All tests passing; system works smoothly on real data

---

Phase 6: Documentation & Report (Weeks 11-12)

Goal: Prepare project report and presentation

☐ **Technical Documentation**

- API documentation
- Database schema explained
- Architecture decisions + reasoning
- Deployment guide

☐ **Project Report**

- Problem statement
- Literature review (existing academic analytics, LLM document parsing)
- Methodology (pipeline, scoring engine, verification approach)
- Results & validation
- Limitations & future work
- Code snippets, architecture diagrams

☐ **Presentation Slides**

- Demo walkthrough
- Architecture overview
- Key findings
- Future work

Deliverable: Final report + presentation ready for viva/defense

---

TEAM ROLES (SUGGESTED)

Role	Responsibilities
Lead / Project Manager	Overall timeline, integration, API design
Backend Developer	API, database, LLM integration, scoring engine
Frontend Developer	UI/UX, dashboards, verification screen
ML/Data Engineer	Feature engineering, ML models, EDA
QA / Testing	Test cases, bug reports, documentation

Note: Roles can overlap. In a smaller team, one person might do backend + ML, etc.

DELIVERABLES

Code

- [ ] Source code (organized by module: backend, frontend, ML)
- [ ] README with setup instructions
- [ ] API documentation
- [ ] Database schema
- [ ] Deployment configuration

Documentation

- [ ] Project report (methodology, results, limitations)
- [ ] Architecture & design decisions document
- [ ] User manual (how students use the system)
- [ ] API documentation

Demo

- [ ] Working web application
- [ ] End-to-end demo on real syllabus + papers
- [ ] Presentation slides

Data

- [ ] Sample syllabus PDF
- [ ] Sample question papers
- [ ] Example output dashboards (screenshots)

SUCCESS CRITERIA

Minimum (Must Have)

- ✅ System can parse syllabus PDF and extract topics
- ✅ System can parse question papers and map questions to topics with human verification
- ✅ Students can enter marks and see topic-wise accuracy
- ✅ Rule-based scoring engine works and is transparent
- ✅ Personalized recommendations are generated

Good-to-Have

- ✅ Trend analysis across multiple exams
- ✅ ML models trained and showing predictions (labeled experimental)
- ✅ EDA dashboards with visualizations
- ✅ Clean, intuitive UI

Nice-to-Have

- ✅ Mobile-responsive design
- ✅ Export analytics as PDF report
- ✅ Data validation on OULAD dataset
- ✅ Advanced visualizations

TIMELINE & MILESTONES

Week	Phase	Milestone
1-2	Phase 1: Foundations	Backend + frontend skeleton; LLM working on test PDFs
3-4	Phase 2: PDF Pipeline	Full parsing pipeline + verification screen working
5-6	Phase 3: Analytics	Dashboard complete; all scoring & recommendations working
7-8	Phase 4: ML & EDA	ML models trained; experimental module integrated
9-10	Phase 5: Testing	All tests passing; real data validated
11-12	Phase 6: Documentation	Report, presentation, final code cleanup

---

RISKS & MITIGATIONS

Risk	Impact	Mitigation
LLM parsing fails on messy PDFs	High	Use multimodal LLM (Claude/GPT-4V); add human verification loop; test early on real PDFs
Question-to-topic mapping is ambiguous	Medium	Use weighted multi-topic mapping; human verification screen catches errors
No training data for ML validation	Medium	Use synthetic data + feature validation against OULAD; label as experimental
Timeline overruns	High	Prioritize core features first; defer "nice-to-have" features to future work
Database schema changes mid-project	Medium	Design schema carefully in Phase 1; use migrations for any changes

---

FREQUENTLY ASKED QUESTIONS

Q: Why is ML optional/experimental? A: Because we don't have real longitudinal student data to train on. Using synthetic data alone isn't reliable enough for a primary output. The novel part of this project is the document parsing pipeline, not the ML prediction.

Q: Why not use existing academic analytics systems? A: Existing systems (Moodle, Canvas, etc.) assume topic tagging is already done. They don't solve the hard problem of extracting structured data from raw PDFs. We're automating that step.

Q: What if the LLM gets the topic mapping wrong? A: That's why we have the human verification screen. Students review and correct AI suggestions. It's a practical workaround that doesn't require the LLM to be perfect.

Q: Can this work with multiple universities' syllabus formats? A: Not in this version—we're targeting one format (your university's). Generalizing to multiple formats is future work.

Q: Is this a mobile app? A: No, web-first only. Mobile app is future work.

Q: Who can use this system? A: Individual students. No multi-user, no classroom views, no teacher dashboards in this version.

HOW TO USE THIS DOCUMENT

1. **For team onboarding:** Share this entire document with all team members

2. **For progress tracking:** Update milestone checkboxes as you complete work
3. **For scope discussions:** Refer to "In Scope vs Out of Scope" section to say no to feature creep
4. **For viva prep:** Keep this alongside your detailed project vision doc (the 27-section master doc)

Last Updated: [Today's Date] **Project Status:** Planning Phase **Next Step:** Finalize tech stack and assign team roles